

AGENTIC AI SOLUTIONS ENGINEER — WAREHOUSE AUTOMATION

Reflection

Warehouse Ops — Order Exception Resolver + Shift Planner

Repository	https://github.com/sunkaramahesh09/warehouse-agent
Live app	https://app-production-fd3e.up.railway.app
Source	<code>docs/REFLECTION.md</code>
Generated	2026-10-01

Simulated prototype: all data is fictional; nothing connects to a real warehouse, carrier or customer system.

Reflection

Key design decisions

1. **The LLM proposes; deterministic code disposes.** The model selects investigation tools and writes explanations. A pure `assessment` recomputes the facts from tool results, and a pure `guard` enforces SOP-APR-001 and chooses the *more conservative* of proposal and assessment. I considered letting the LLM decide within limits, but a take-home reviewer (and a warehouse) needs outcomes that are safe by construction, not safe on average. The cost: when the model and the rules disagree, the rules win, so the LLM's leverage is in *investigation and explanation*, not authority. That is intentional.
2. **Shared SOP with machine-readable parameters.** Policies are text for citation *and* parameters for enforcement (stale threshold, duplicate window, action lists, urgency buckets, travel minutes, approval TTL). Both workflows read the same rows. This avoids the "two hardcoded rule sets" trap the PRD warns about.
3. **Every mutation defends itself.** Action tools re-check business rules inside the transaction (for example, `sync_order_status` re-verifies all three SOP-SOT-002 conditions), so even a guard bug cannot produce an illegal state change. Idempotency keys make retries safe.
4. **Simulated clock.** All ages, deadlines, expiries and execution progress use `sim_state.sim_now`. Scenarios are reproducible, and "mid-shift" is something you can actually demonstrate.
5. **Planner as a pure function.** `buildPlan(snapshot, previous)` has no I/O, which made it straightforward to unit-test capacity, skills, deadlines, determinism and incremental replanning. Replanning re-runs the same policy with *stickiness* to the previous picker, freezes completed and valid in-progress work, and diffs against the previous version to explain every change.
6. **The planner consumes resolver decisions, not raw exceptions.** The planner blocks orders that are ON_HOLD and warns on untriaged exceptions. It does not reinterpret exception records itself. Triage (is the shortfall real? does the stale shipment even affect picking?) is the resolver's job. The trade-off: before triage, ORD-1004 is schedulable with a visible warning. I chose this over "any open exception blocks work" because that rule would block orders on exceptions that don't affect picking (for example, stale outbound shipments).
7. **One deployable.** Fastify serves the built React app, so there is no CORS and one URL. PostgreSQL gives transactions, row locks, generated columns and partial unique indexes that directly implement safety properties.

AI assistance disclosure

This project was built with substantial help from **Claude Code (Anthropic)** as a coding assistant. It was used for: - reading the PRD and drafting the requirements matrix and Phase 0 design, - generating most of the TypeScript (server, planner, resolver, tools, UI), tests, and documentation, - running the tests and scenario suites, visually checking the UI in a headless browser, and debugging.

Runtime AI: the Exception Resolver uses Google Gemini (`gemini-3.1-flash-lite` , free tier) through an OpenAI-compatible API to select investigation tools and write explanations. The planner can optionally use it to explain plans.

I remain responsible for the requirements interpretation, architecture, safety boundaries, validation of behaviour, and final decisions. Every claim in these docs is backed by a test, a scenario result, or an audit example in `docs/results/`.

Changes from Phase 0 (as implemented)

Phase 0 plan	What changed	Why
Suggested multi-package monorepo (packages/*)	Two workspaces (server , web) with folder layering	Same boundaries, less build plumbing within the time box
Drizzle/Prisma suggested	Raw SQL via pg	Safety-critical statements (row locks, atomic approval claim, partial unique indexes) stay explicit. No codegen
Planner reads cycle counts directly	Planner consumes the resolver's hold instead (decision 6)	Keeps triage in one place and makes the cross-agent effect observable
Picker capacity = fixed minutes	Added pickers.consumed_minutes	Replanning mid-shift needs remaining capacity once work has been done
Sequence numbers restarted after completed work	Per-picker sequence continues after completed items	Unchanged assignments were wrongly reported as RESEQUENCED (found by a unit test)
Blocked → blocked with new numbers logged as NEWLY_BLOCKED	Reported as UNCHANGED with updated details	The change log should list real changes only
Default model gemini-2.5-flash	gemini-3.1-flash-lite	2.5 is retired for new keys. gemini-3.5-flash free tier allows only 20 requests
—	Added claimed-action detector	Gemini wrote "I have placed the order on hold" before anything executed, even when told not to

Debugging lesson

A green test suite hid that the LLM path never ran. My first LLM runs "worked" but every investigation ended in llm->deterministic fallback, and the scenarios still passed. The green test suite hid that the LLM path wasn't being exercised. Two separate root causes emerged: the default model returned 404 (retired for new keys), and after switching, the free-tier quota of gemini-3.5-flash was 20 requests. Each fallback was visible in the run report (fallback_reason) and audited as LLM_FALLBACK , which is how I found them. I fixed the model choice, honoured the provider's retry in Ns hint, preserved the provider's extra_content (Gemini 3 thought signatures, which its docs require to be echoed back on multi-turn tool calls), tightened the prompt, and **added the per-run agent mode to the scenario results**, so "18/18 PASS" in LLM mode now proves every run was actually LLM-driven. Lesson: when a system has a safe fallback, test results must also report which path ran, or the fallback will quietly mask the feature you think you are testing.

Optional features: decisions

- **Outbox, not callbacks.** Events are written in the same transaction as the change, so a crash can't produce a state change without its event, or an event without a change. The dispatcher runs after each request instead of in a background worker, which is simpler and deterministic for a single instance, at the cost of a slower request when automation is on.
- **Automation reuses, never bypasses.** Auto-investigation calls the same guarded orchestrator; auto-replan calls the same planner. Nothing new can approve.
- **Churn above makespan in the optimizer objective.** Mid-shift, stability for pickers beats a slightly shorter makespan. The optimizer only reshuffles a live plan to reduce SLA risk or lateness.

- **Evaluation measures the guarded outcome *and* the raw model.** Accuracy of the final decision and agreement of the model's proposal are reported separately. High accuracy with lower agreement shows the guard doing its job, not a model that is always right.
- **Skipped browser automation** (see [OPTIONAL_FEATURES.md](#)).

A second instance of the same lesson

While building the optional features, I ran the LLM evaluation, the LLM scenario suite and a live browser test back-to-back on one free-tier key. The suite still reported "20/20 PASS", but its per-run agent-mode line showed **8 runs had fallen back** to deterministic (429 quota exceeded , a per-minute limit). Because the results record which path ran, I didn't publish that as LLM evidence. I added client-side request pacing (`LLM_MIN_INTERVAL_MS` , also set on Railway) and reran the suite with the key otherwise idle. The evaluation report also shows its fallback rate honestly (1 of 14 runs).

Remaining risks and limitations

- **Header-based roles, no authentication.** Anyone with the URL can act as Operator. This is acceptable for a simulated prototype, but not for production.
- **One shared demo database.** Concurrent reviewers share state, and a reset affects everyone. Mutations are serialised in-process, which assumes a single server instance.
- **LLM variability and quota.** Tool sequences vary between runs. The free tier can rate-limit, in which case runs fall back to deterministic (transparently). Latency is 10–40 s per investigation.
- **The assessment encodes which policy governs which exception type.** The policy *content and parameters* are shared data, but the mapping from exception type to required policy lives in code. New exception types need code, not just a new SOP.
- **Planner optimality.** The default greedy planner is not globally optimal; the opt-in local search (relocate/swap) improves sequencing but is still a heuristic. Neither strategy splits orders across pickers, and travel time is a simple zone-distance model.
- **Simulated execution** (`advance_clock`) models picking progress linearly, and there is no packing/shipping simulation.
- Scenario checks cover the seeded cases, not arbitrary data.

Next improvements

Event-driven triggers (cycle-count ingestion → detection → auto-investigation → auto-replan), a metrics/evaluation dashboard, and local-search scheduling were on the original list and have since been built (see [OPTIONAL_FEATURES.md](#)). What remains:

1. Move the event dispatcher to a background worker with retries/backoff if automation must not add request latency.
2. Real authentication plus per-role permissions, and per-reviewer sandboxes (a database schema per session) so reviewers don't collide.
3. An approval-gated *release hold* and *inventory adjustment* flow initiated from the escalation card.
4. Order splitting across pickers when the SOP allows it, and a richer travel model, behind the same pure planner interface.
5. Longer-horizon evaluation: many LLM runs across models and prompts, with cost and latency tracking per run.